

IMPORTER END-TO-END ROLE-STAGE MATRIX

Multi Tenant Platform Build - canonical importer flow from order creation to receipt in Ghana

Revision 7 - non-blocking operational progression even when platform funding recognition is still pending

This revision supersedes Revision 6 and updates the importer baseline so late staff funding recognition does not bring a genuinely purchased order to a standstill. The control model now distinguishes between operational purchase reality and platform funding confirmation.

Role definition

The importer is the commercial order owner and beneficiary of funded purchases, downstream fulfilment, and carried-forward credit outcomes. The importer creates and submits the order, provides retailer evidence after purchase, operates the OCR reconciliation workspace, uses AI-assisted retailer communication for child exceptions, tracks later progress to Ghana, confirms receipt, and raises downstream issues. The importer does not upload DVA/card statements, does not set FX rates, and does not perform internal funding reconciliation controls.

Locked architecture, schema, and implementation assumptions used here

- Importer creates the order; supervisor and admin are staff for DVA upload, FX setup, statement review, funding suggestion review, reconciliation, and credit application.
- Statement belongs to the importer; upload and reconciliation of DVA/card statements are staff-only actions.
- Each importer operates in one currency. FX is set upstream and used to normalize local statement lines into GBP before funding reconciliation.
- Courier lookup exists through couriers. Supplier invoice upload / invoice reference / OCR lines / manual lines exist through supplier_invoices and supplier_invoice_lines.
- order_screenshots and order_tracking_submissions exist in the live database and canonical SQL source. orders.screenshot_url remains legacy / backward-compat only.
- dispute_lines now carries conversation_status and intended_remedy. dispute_messages now carries in_reply_to_message_id, ai_input_context_json, ai_model_used, and ai_prompt_hash. status_transitions now supports entity_type = dispute_line.
- Tracking and invoice are independent child submissions of the same order. Tracking can come first, invoice can come first, or both can arrive close together.
- Correct lines may progress while unresolved lines become child exceptions. The parent order is only fully cleared once progressed lines plus resolved child outcomes reconcile back to the original submitted quantity and value.
- Importer is the primary user of the OCR reconciliation workspace in the normal path.
- Retailer exception handling uses a server-side AI drafting service (Claude / ChatGPT-style integration) that must take retailer SOP, current child-exception status, chosen path (refund or replacement), intended_remedy, and prior retailer conversation context as inputs.
- Refund progression requires supervisor/admin approval before the importer can proceed with the refund communication path. Until approved, the refund branch is blocked / greyed out and cannot progress as if approved.
- The remaining build gaps are app wiring, UI gating, status-transition enforcement in the app/backend, and AI-service implementation. The database/control model itself is now in place.

Non-blocking operational progression rule

- Operational purchase reality and platform funding confirmation are related but not identical.
- If the retailer invoice exists, the retailer purchase happened. That does not by itself prove that staff have already matched the order to DVA/card funding inside the platform.
- The platform should therefore allow operational work to continue when reality has moved ahead of staff matching.

- Allowed before funding match: tracking submission, invoice upload, invoice reference submission, OCR/reconciliation work, child-exception creation, AI-assisted retailer communication, ready-for-shipment handoff, and shipper/logistics progression where goods are genuinely moving.
- Blocked until funding match: platform-funded confirmation, final parent full-clearance / completion, final refund / payout / carried-credit settlement, and final accounting / VAT release.
- This avoids operational standstill without weakening funding controls.

Importer operating boundary

- Importer does: create orders; upload multiple retailer screenshots; enter category lines and declared totals; view quote; receive order ref and authorisation ID; submit tracking and/or invoice evidence in either order; operate the OCR reconciliation workspace; bulk progress correct lines; add and delete only manual exception lines; select child-exception remedy intent; use AI-generated retailer drafts in the retailer account; paste retailer replies back for next AI drafts; follow progress to Ghana; confirm receipt; raise issues.
- Importer does not: upload DVA/card statements; set FX rates; reconcile DVA lines; approve internal funding matches; apply importer credit as an internal control action; approve refunds; bypass the retailer communication status machine; delete OCR source lines; perform shipper-side booking/evidence actions.

Child exception state machine (canonical)

- `child_exception_created` - unresolved line split from the parent after importer reconciliation.
- `remedy_selected` - intent has been chosen at child level; `intended_remedy` distinguishes refund vs replacement.
- `refund_pending_approval` - `intended_remedy` = refund and supervisor/admin approval not yet granted.
- `retailer_draft_ready` - AI has generated the next message draft using SOP + status + prior context. This can follow replacement directly from `remedy_selected`, or refund only after approval.
- `retailer_contacted` - importer has used the draft in the retailer account and is waiting.
- `retailer_response_received` - importer pasted the retailer reply back into the system.
- `ai_next_draft_ready` - AI generated the next response based on the reply, SOP, intended remedy, and current status.
- `awaiting_retailer_resolution` - conversation has reached a point where the retailer outcome is pending or being finalized.
- `resolved_refund` / `resolved_replacement` / `resolved_credit` / `closed_no_action` - child outcome complete and ready to reconcile back to the parent.

Stage 1 - Onboarding and account presence

Role: Importer

Allowed action: Exist in the platform as an onboarded trading entity linked to one shipper and one operating country/currency context.

Resource(s) / system(s) used: Importer-facing Next.js/Vercel account surfaces; Supabase importer, country, currency, shipper, and role-auth mappings.

Data touched: Importer master data, country, shipper link, operating currency, commercial identifiers, and login/access linkage.

Validation rules: Importer must be active, tied to the correct shipper, and operate in a single currency context.

State transition: Importer becomes eligible to create and view orders.

Next actor: Importer

Exception path: Wrong shipper, wrong country, or inactive setup must be corrected before operational use.

Stage 2 - Create order from multiple retailer screenshots

Role: Importer

Allowed action: Start a new order by uploading one or more retailer screenshots and opening the order shell that will become the parent order.

Resource(s) / system(s) used: Next.js/Vercel importer order-entry page; Supabase orders + order_screenshots; image compression before storage.

Data touched: Parent orders row, order_screenshots child rows, importer/retailer/shipper context, destination hub, initial status, and audit timestamps.

Validation rules: Importer can only create for themselves; screenshots should attach to the correct order; order opens unfunded.

State transition: Order shell exists and is ready for commercial line entry.

Next actor: Importer

Exception path: If screenshots are missing or attached to the wrong order, the order should be corrected before submission.

Stage 3 - Enter category lines, quantity, and declared value

Role: Importer

Allowed action: Add category-based lines, each with quantity and value, until the total submitted order quantity and value are complete.

Resource(s) / system(s) used: Next.js/Vercel line-entry forms; Supabase order_category_lines; order totals logic.

Data touched: order_category_lines.markup_category_id, qty, amount_inc_vat_gbp, markup fields, and rolled-up orders.total_qty_declared / orders.order_total_gbp_declared.

Validation rules: No orphan lines; totals must roll up cleanly; values must be positive and commercially sensible.

State transition: The order has an original submitted parent baseline of quantity and value.

Next actor: Importer

Exception path: If category lines and header totals diverge, hold for correction rather than passing a bad parent baseline downstream.

Stage 4 - Review quote and receive identifiers

Role: Importer

Allowed action: Review the quote and the local-currency context, then submit the order into the funding queue.

Resource(s) / system(s) used: Next.js/Vercel quote UI; Supabase order quote fields; FX rates used upstream for quote display.

Data touched: Quote-side fields and generated identifiers: order_ref and payment_auth_id (authorisation/payment reference).

Validation rules: payment_auth_id must be unique per order; quote display must not distort the Day 2 purchase threshold rule (declared goods plus markup, shipping excluded).

State transition: Order is submitted into pending_dva_funding with funded_at = null.

Next actor: Supervisor/Admin staff for funding recognition; importer can also continue operational evidence capture later when retailer evidence exists.

Exception path: If the quote is inconsistent with current internal rates, staff correct the rates upstream rather than allowing a bad quote to persist.

Stage 5 - Monitor funding recognition without freezing operations

Role: Importer

Allowed action: Monitor whether staff have matched platform funding while continuing to work on operational evidence and downstream order handling where reality has already moved on.

Resource(s) / system(s) used: Importer dashboard funding views; Supabase order_funding_position_vw and related summaries; later downstream dashboard indicators.

Data touched: Funding threshold, funded total, gap remaining, funded flag, funded timestamp, order status, and any platform-funded confirmation markers.

Validation rules: Importer has read-only visibility to the funding lane and does not reconcile DVA lines or apply credit themselves. Funding recognition can lag without blocking evidence capture or genuine shipment progression, but final completion and final financial settlement remain blocked until funding is confirmed.

State transition: Order may remain operationally active while still platform-unfunded; later, once staff-driven funding reaches threshold, the order becomes funded exactly once.

Next actor: Importer plus supervisor/admin in parallel lanes

Exception path: If importer says 'I paid already' but the order remains open, staff investigate DVA statement ownership, auth match, FX normalization, and prior reconciliations. The importer should not be forced into operational standstill while that investigation happens.

Stage 6 - Dynamic post-purchase submission (tracking and invoice can arrive in either order, before or after funding match)

Role: Importer

Allowed action: From the dashboard, submit post-purchase retailer evidence in whatever order reality allows: tracking details first, retailer invoice first, or both close together, even if staff funding recognition is still pending.

Resource(s) / system(s) used: Next.js/Vercel importer dashboard; Supabase supplier_invoices; couriers lookup; order_tracking_submissions child rows under the order for pre-invoice or post-invoice tracking submissions.

Data touched: Tracking branch: courier_id, tracking_ref, tracking_date, optional tracking screenshot, submit timestamp. Invoice branch: supplier_invoices.invoice_ref, supplier_invoices.invoice_pdf_url, uploaded_by_operator_id, and linked order_id.

Validation rules: The flow must not force invoice before tracking or tracking before invoice; both are independent children of the same order. Courier choice is mandatory so later actors know which courier site/template to use for the tracking reference. Funding-match delay must not block these submissions.

State transition: Order moves into a tracking-only, invoice-only, or tracking-and-invoice-received state independently of whether platform funding has already been matched.

Next actor: Importer continues once the invoice branch is present; shipper/internal staff can also use the tracking branch earlier.

Exception path: If the wrong courier is selected, the tracking reference becomes operationally unusable; if invoice is delayed, tracking can still be submitted without blocking the dashboard workflow.

Stage 7 - Importer OCR reconciliation workspace

Role: Importer

Allowed action: Open the reconciliation workspace for the order once the retailer invoice exists, even if staff funding recognition is still pending. Review original screenshots, original submitted totals, uploaded invoice, and OCR-extracted lines together.

Resource(s) / system(s) used: Next.js/Vercel reconciliation UI; Supabase supplier_invoices, supplier_invoice_lines, order_reconciliation_vw; Mindee OCR output surfaced into the workspace; image/PDF assets loaded from storage.

Data touched: Original order screenshots, original parent order quantity/value, invoice PDF, supplier_invoice_lines, OCR metadata, and any manual exception lines.

Validation rules: The importer is an active workspace user here. This is not a staff-only invoice workspace. OCR review is allowed before platform funding recognition is complete because the invoice proves the retailer purchase happened.

State transition: Importer can now separate clean invoiceable lines from unresolved exceptions.

Next actor: Importer

Exception path: If OCR extraction is bad or incomplete, importer still works from the OCR output but uses edits/manual lines to get to a reconcilable state.

Stage 8 - Edit OCR lines and manage manual exception lines

Role: Importer

Allowed action: Edit OCR-extracted lines for size, quantity, and value; add manual lines for missing items or exceptions; delete only manually added lines.

Resource(s) / system(s) used: Importer reconciliation UI; Supabase supplier_invoice_lines; line-source controls (ocr_extracted | manually_added).

Data touched: OCR lines with editable commercial fields and manual exception lines.

Validation rules: OCR lines are editable but not deletable. Only manual lines may be deleted. Description/source provenance of OCR lines must remain preserved for audit. Manual lines must remain distinguishable from OCR lines.

State transition: Workspace becomes a controlled mixed set of corrected OCR lines plus manual exception lines.

Next actor: Importer

Exception path: If importer tries to remove an OCR source line, the system must block it; only manual-added lines can be removed.

Stage 9 - Bulk progress correct lines and split child exceptions

Role: Importer

Allowed action: Bulk submit or confirm the lines that are correct so they can progress, while leaving unresolved or missing lines in the exception path.

Resource(s) / system(s) used: Importer reconciliation UI; Supabase supplier_invoice_lines, disputes/exception structures, later invoiceable subset release logic.

Data touched: Correct lines marked confirmed/progressed; unresolved/missing lines represented as manual exception children or later dispute-linked children; order reconciliation views.

Validation rules: The whole order does not have to wait if some lines are correct. Correct lines can progress.

Unresolved lines split out into child exception/replacement/refund units. Together they must still reconcile back to the original parent order's quantity and total value. Funding-match delay must not by itself freeze this operational split.

State transition: Good lines move forward to the next operational stage; child exceptions stay open separately.

Next actor: Internal purchasing/operations and later shipper-side logistics flow for progressed lines; exception handling continues for the child lines.

Exception path: If the progressed set plus unresolved children no longer reconciles to the original parent order baseline, the order should remain visibly not fully cleared.

Stage 10 - View purchase progression and ready-for-shipment handoff even if funding recognition is still pending

Role: Importer

Allowed action: See the progressed subset move into ready-for-shipment states while child exceptions may still remain open and staff funding recognition may still be catching up.

Resource(s) / system(s) used: Importer dashboard progress views; Supabase downstream status fields, invoiceable subset indicators, operational summaries, and shipper handoff visibility.

Data touched: Retailer purchase progress, invoiceable subset release state, progressed quantities/values, unresolved child-exception visibility, shipper handoff markers, and any funding-pending warning flags.

Validation rules: This handoff must remain explicit. Correct lines may move into the shipper lane while unresolved children remain separate. Open child exceptions do not erase the shipper handoff for the progressed subset, and late staff funding recognition should not falsely halt a genuinely moving order. However, the parent cannot be fully cleared while funding is still unconfirmed.

State transition: Progressed lines move toward shipper receipt, booking, and shipment, while child exceptions continue on their own governed track.

Next actor: Shipper and internal staff for later logistics execution; importer continues on child-exception steps where needed.

Exception path: Supplier shortages, unavailable lines, or unresolved children continue in the exception branch without blocking all already-correct lines from reaching the shipper lane.

Stage 11 - Select remedy intent for each child exception

Role: Importer

Allowed action: For each child exception, choose the intended commercial path: refund or replacement.

Resource(s) / system(s) used: Importer exception workspace; Supabase dispute / child-exception records; SOP lookups; status model built from conversation_status + intended_remedy.

Data touched: Child exception status, intended_remedy, retailer context, amount impact, child-to-parent linkage.

Validation rules: Selection must happen at the child level, not as a vague parent-order note. The chosen intent must remain traceable and status-driven. In the actual schema, conversation_status moves to remedy_selected and intended_remedy holds refund or replacement.

State transition: Child exception enters conversation_status = remedy_selected, with intended_remedy = refund or replacement.

Next actor: Supervisor/Admin for approval only if refund is selected; importer can continue once the allowed path is open.

Exception path: If the wrong path is chosen, it must be corrected before retailer communication starts; otherwise the AI will draft against the wrong SOP and status.

Stage 12 - Refund approval gate before retailer communication

Role: Importer

Allowed action: If refund is selected, wait for supervisor/admin approval before progressing the retailer communication lane. If replacement is selected, proceed under the replacement path without the refund-specific gate.

Resource(s) / system(s) used: Importer dashboard; supervisor/admin approval controls; Supabase disputes approval fields plus dispute_line status transitions.

Data touched: refund_approved_by_staff_id, refund_approved_at, conversation_status, intended_remedy, and lock/disabled UI state.

Validation rules: Refund path must be greyed out / blocked until approval is granted. No retailer-facing refund progression should occur as if approved before that gate is passed. Replacement bypasses only the refund-specific approval gate; it still remains inside the child-exception status machine. Even if the retailer has already effectively acknowledged the issue, final refund settlement still cannot complete until platform funding is recognized.

State transition: Refund branch moves from `remedy_selected` to `refund_pending_approval` and then to `retailer_draft_ready` only after approval. Replacement can move from `remedy_selected` to `retailer_draft_ready` without the refund gate.

Next actor: Importer once an allowed communication state exists.

Exception path: If refund is not approved, the importer cannot progress the refund lane. Supervisor/admin may request more info, reject the path, or redirect to another outcome.

Stage 13 - Use AI-assisted retailer communication loop

Role: Importer

Allowed action: Use the retailer account and the AI-generated drafts to contact the retailer about the child exception, then paste retailer replies back so the AI can generate the next response until the child is resolved.

Resource(s) / system(s) used: Next.js/Vercel exception communication UI; server-side AI drafting service (Claude / ChatGPT-style integration); retailer SOPs; stored child-exception status; pasted retailer replies; `dispute_messages` audit/threading fields.

Data touched: Chosen remedy path, retailer SOP, current `conversation_status`, `intended_remedy`, prior retailer messages, pasted retailer response, generated next draft, sent/not-sent markers, `in_reply_to_message_id`, `ai_input_context_json`, `ai_model_used`, and `ai_prompt_hash`.

Validation rules: AI cannot be generic. Drafting must be status-driven and SOP-driven. Inputs must include refund vs replacement path, `intended_remedy`, current status, child facts, and prior conversation context. Importer uses the draft in the retailer account; the retailer reply is pasted back for the next AI step. This loop can continue operationally before staff funding recognition, but final financial closure cannot complete until funding is confirmed.

State transition: Child moves through `retailer_draft_ready`, `retailer_contacted`, `retailer_response_received`, `ai_next_draft_ready`, and `awaiting_retailer_resolution` until a final outcome is achieved.

Next actor: Importer continues the loop; supervisor/admin monitor approval/control aspects; shipper only enters if liability becomes shipper-side.

Exception path: If the importer pastes the wrong retailer response, uses the wrong retailer account context, or bypasses the status machine, the communication trail becomes unreliable and should be corrected before final outcome is booked.

Stage 14 - Resolve child outcome and reconcile back to parent

Role: Importer

Allowed action: See the child exception resolve into refund, replacement, carried credit, payout visibility where applicable, or closure, with the result traced back to the parent order baseline.

Resource(s) / system(s) used: Importer status pages; Supabase disputes, child-order references, `importer_credit_ledger`, refund/credit note state; downstream Sage logic remains indirect.

Data touched: Resolved child status, replacement child order references, carried credit balance, refund outcome, payout visibility, and parent-order partial/fully-cleared state.

Validation rules: A child can resolve separately, but the parent is only fully cleared once progressed lines plus all resolved children reconcile back to the original parent quantity/value. If staff funding recognition is still pending, the child can be operationally resolved, but final financial settlement and full parent completion remain blocked.

State transition: Parent remains partially cleared or becomes fully cleared, depending on whether all child outcomes are now complete and funding is now confirmed.

Next actor: Shipper/internal staff for progressed logistics; supervisor/admin for remaining financial/governance work if needed.

Exception path: If the child resolves but the parent baseline still does not reconcile, or if funding is still not confirmed, the system must not silently mark the parent fully cleared.

Stage 15 - Track movement to Ghana and later receipt

Role: Importer

Allowed action: Monitor the order's logistics progression, including shipper receipt, booking, shipment movement, and arrival/receipt in Ghana.

Resource(s) / system(s) used: Importer-facing order tracking/status UI; Supabase operational status records; courier/tracking submission records; shipper-generated evidence links.

Data touched: Shipment status, receipt status, booking references, tracking references, selected courier context, and later delivery/arrival markers.

Validation rules: Importer does not perform shipper-side booking/evidence actions, but they do need the right courier context to interpret the tracking reference submitted earlier. Movement can continue even if staff funding recognition is late.

State transition: Order reaches receipt/delivery readiness in Ghana.

Next actor: Importer confirms receipt or raises a discrepancy.

Exception path: If movement stalls, tracking is invalid, or received goods are incomplete, the path shifts into discrepancy/dispute handling.

Stage 16 - Confirm receipt in Ghana or report discrepancy

Role: Importer

Allowed action: Confirm successful receipt in Ghana or report what is wrong: missing items, damage, wrong goods, under-delivery, or non-delivery.

Resource(s) / system(s) used: Next.js/Vercel status and discrepancy UI; Supabase disputes and downstream dispute-line structures.

Data touched: Receipt confirmation state, dispute type, comments, amount impact, desired outcome, supporting notes/evidence.

Validation rules: Importer should only act on their own orders. Dispute initiation must capture enough detail for staff review and later accounting treatment.

State transition: Successful receipt pushes toward completion readiness. Reported issue moves the order/child exception into a dispute or replacement/refund branch.

Next actor: Supervisor/Admin staff review and decide refund, replacement, credit, or closure; shipper may also be involved depending on stage and liability.

Exception path: Late or vague importer reporting may require additional evidence before any refund/replacement decision.

Stage 17 - View refund / replacement / credit outcome

Role: Importer

Allowed action: See the downstream result of the issue path: refund, replacement child order, carried credit, payout result where applicable, or closure.

Resource(s) / system(s) used: Importer-facing status pages; Supabase disputes, importer_credit_ledger, refund/credit note state; Sage Cloud API remains downstream and indirect.

Data touched: Dispute status, refund approval state, replacement child order references, carried credit balance, payout visibility, and downstream outcome fields.

Validation rules: Importer views the outcome but does not approve internal accounting postings. Operational visibility can exist before platform funding is matched, but final settlement remains blocked until funding is confirmed.

State transition: Issue resolves into refund, replacement, carried credit, payout visibility, or closed state, subject to the funding-confirmation gate on final settlement.

Next actor: If credit is created, staff may later apply it to another importer order; if replacement is approved, lifecycle restarts on the child order.

Exception path: If importer disputes the outcome amount, staff must trace the original discrepancy, supplier outcome, approval status, AI communication trail, and credit/refund entries.

Stage 18 - Reuse carried-forward credit on later order

Role: Importer

Allowed action: Indirectly benefit from available importer credit on a future order.

Resource(s) / system(s) used: Importer balance visibility UI; internal credit-application workflow using importer_credit_ledger and funding functions behind the scenes.

Data touched: importer_balance_vw, importer_credit_ledger, target orders, and order funding events.

Validation rules: Unapplied credit does not fund an order until staff applies it; amount cannot exceed available credit or target order gap.

State transition: Available balance reduces; later order funding improves or reaches funded state.

Next actor: Supervisor/Admin staff remain the operational actor for the application action.

Exception path: If importer believes credit exists but it is not available, staff check whether it is pending payout, already applied, or still tied to another exception branch.

Stage 19 - Completion and archive

Role: Importer

Allowed action: View the final completed/closed order state and refer back to historical records if needed.

Resource(s) / system(s) used: Importer-facing order history UI; Supabase archived/historical status reads; downstream accounting records may exist in Sage but are not importer-operated.

Data touched: Completed order state, funded history, receipt/dispute outcome, and historical references.

Validation rules: Historical reads remain role-controlled and immutable funding history should not be silently rewritten. Final completion / full clearance is only allowed once operational progression, child outcomes, and platform funding recognition are all complete.

State transition: Order leaves active operational work unless reopened through a controlled exception path.

Next actor: No active actor in the happy path.

Exception path: If a post-completion issue emerges, staff reopen through controlled dispute/exception handling instead of editing historical truth directly.

Importer happy-path summary

- Importer is onboarded in the correct shipper and country/currency context.
- Importer uploads retailer screenshots, creates the parent order, enters category lines, and receives the order reference plus authorisation/payment reference.
- Staff funding recognition may lag, but the importer is not frozen while that happens.
- After retailer purchase, importer can submit tracking details first, invoice first, or both in either order.
- Once invoice exists, importer enters the OCR reconciliation workspace, edits OCR line size/quantity/value where needed, adds manual missing-item lines, deletes only manual lines, and bulk progresses the correct lines.
- Progressed lines enter the explicit ready-for-shipment handoff and can move into the shipper lane while unresolved lines remain child exceptions.
- Importer selects refund or replacement for each child. If refund is selected, the refund branch is blocked until supervisor/admin approval.

- Once allowed, the importer uses AI-generated retailer drafts, sends them through the retailer account, pastes replies back, and continues the loop until the child resolves.
- Resolved children must still reconcile back to the original parent order baseline before the parent is fully cleared.
- Final completion, final financial settlement, and final accounting/VAT release still wait for platform funding confirmation.
- Importer follows shipment movement to Ghana, confirms receipt, or raises issues.

Importer exception-path summary

- Payment made but order still open: staff investigate DVA statement ownership, auth match, FX normalization, and reconciliation state.
- Tracking before invoice or invoice before tracking: both are valid; the system must not force a rigid sequence.
- Wrong courier selected: tracking reference becomes operationally ambiguous and must be corrected.
- Bad OCR extraction: importer edits OCR lines, adds manual lines, and isolates unresolved child exceptions rather than deleting OCR source lines.
- Partial correctness: clean lines progress; child exceptions remain open; the parent order remains only partially cleared until final reconciliation is complete.
- Refund selected: no retailer refund progression until supervisor/admin approval is granted.
- Retailer communication loop: every AI draft must be status-aware, intended-remedy-aware, SOP-aware, and based on the prior pasted retailer response.
- Funding still pending: operational progression can continue, but final settlement and final completion remain blocked.
- Goods missing or damaged on receipt in Ghana: importer raises discrepancy and staff govern refund/replacement/credit outcomes.

Order-to-receipt validation checks

Simulation A - Tracking arrives before invoice

- Importer creates order from screenshots, category lines, and declared totals; quote, order ref, and authorisation ID are generated.
- Staff funding recognition may still be pending.
- Importer receives courier/tracking information before any formal invoice exists and submits courier + tracking ref + tracking date.
- Later the retailer invoice becomes available; importer uploads invoice and invoice reference.
- Importer opens the OCR workspace, confirms correct lines, and creates manual child exceptions for any missing items.
- If a child needs refund, the refund lane waits for supervisor/admin approval before the retailer communication loop begins.
- AI generates retailer drafts based on child status and SOP; importer pastes retailer replies back for next drafts until the child resolves.
- Validation result: passes.

Simulation B - Invoice arrives before tracking

- Importer creates order and receives order ref/auth ID.
- Invoice is available before a useful tracking reference exists; importer uploads invoice PDF and invoice reference first.
- Mindee OCR output appears in the importer workspace; importer edits OCR line size/qty/value, adds manual lines for missing items, and bulk confirms clean lines.
- Later the retailer exposes tracking details; importer submits courier + tracking ref + tracking date.
- Shipper and internal staff can use the courier context while already-progressed invoiceable lines continue downstream.

- Validation result: passes.

Simulation C - One missing line becomes a child exception

- Order has five original submitted lines.
- Four OCR/manual lines reconcile cleanly and are bulk progressed.
- One missing line becomes a child exception rather than blocking the other four.
- Importer chooses refund or replacement for the child; if refund is selected, approval is required before retailer communication can proceed.
- The child later resolves through refund/replacement/credit, but the parent is only fully cleared when the progressed four plus the resolved child reconcile back to the original parent quantity/value.
- Validation result: passes.

Simulation D - Wrong courier selected

- Importer submits tracking with the wrong courier.
- Funding and invoice reconciliation are unaffected, but logistics interpretation breaks because the tracking reference points to the wrong courier site/template.
- Validation result: confirms courier_id is mandatory and the control is correct.

Simulation E - Payment made but order still open

- Importer created order and paid, but order remains unfunded in the platform because staff have not yet matched the funding line.
- Importer still uploads invoice/tracking and continues operational steps.
- Staff investigate statement ownership, auth match, FX normalization, and reconciliation status. Importer does not reconcile DVA lines themselves.
- Validation result: passes and avoids operational standstill.

Simulation F - Retailer reply loop for replacement

- Importer selects replacement for a child exception.
- AI generates the first retailer draft using retailer SOP and replacement status.
- Importer sends it through the retailer account and later pastes the retailer reply into the system.
- AI generates the next response using the pasted reply plus the current status and intended remedy.
- The loop repeats until replacement is agreed, rejected, or escalated.
- Validation result: passes.

Simulation G - Progressed subset ships while refund child waits for approval

- Importer progresses four clean lines and one child exception remains open with intended_remedy = refund.
- The four clean lines move into the ready-for-shipment handoff and can enter the shipper lane.
- The refund child remains blocked at refund_pending_approval until supervisor/admin approval exists.
- This confirms that explicit shipper handoff and refund gating can coexist without collapsing the parent-child audit trail.
- Validation result: passes.

Schema-fit status

- Already modeled and now implemented: supplier_invoices supports importer/operator invoice upload and invoice reference; supplier_invoice_lines supports OCR-extracted versus manually-added lines; couriers exists for courier lookup; order_screenshots and order_tracking_submissions exist in the live database and canonical SQL source.
- orders.screenshot_url remains in place only for backward compatibility. New screenshot flows should use order_screenshots.

- dispute_lines now supports conversation_status and intended_remedy. dispute_messages now supports in_reply_to_message_id, ai_input_context_json, ai_model_used, and ai_prompt_hash. status_transitions now supports dispute_line conversation flow and refund gating.
- The remaining work is app wiring, UI gating, status-transition enforcement in the app/backend, and AI-assisted exception communication implementation.
- No repo DBML file currently exists. The canonical repo schema source is docs/schema/goodcashback-complete.sql.

Importer resource map by node

Node	Primary resource(s)
Order creation	Next.js/Vercel importer screens; Supabase orders + order_category_lines + order_screenshots; image compression before screenshot storage.
Funding visibility	Supabase read models such as order_funding_position_vw; staff-operated DVA controls remain indirect to the importer.
Tracking submission	Next.js/Vercel importer dashboard; couriers lookup; order_tracking_submissions child records.
Invoice upload	Next.js/Vercel importer dashboard; Supabase supplier_invoices; invoice asset storage.
OCR reconciliation workspace	Mindee OCR output surfaced in Next.js/Vercel; Supabase supplier_invoice_lines, reconciliation views, and manual exception lines.
Ready-for-shipment handoff	Importer dashboard progress views; Supabase progressed subset indicators and shipper handoff status.
Retailer exception communication	Server-side AI drafting service (Claude / ChatGPT-style integration); retailer SOPs; dispute_lines.conversation_status and intended_remedy; dispute_messages threading and AI audit fields; pasted retailer replies; generated next drafts.
Receipt/disputes	Next.js/Vercel importer status and issue screens; Supabase disputes and downstream child-order/credit visibility.
Accounting/compliance downstream	Sage Cloud API, VAT logic, and internal postings remain downstream and indirect from the importer perspective.
Platform delivery (not runtime actor tool)	GitHub and Vercel deployment lifecycle; not importer-operated workflow nodes.

Internal working document - canonical importer role matrix for the Multi Tenant Platform Build (Revision 7)